



AI Knowledge Assistant

A Retrieval-Augmented Generation (RAG) chatbot that answers natural language questions using your own uploaded PDF documents — accurately, transparently, and without hallucination.

PROJECT 5

RAG + DEPLOYMENT

The Problem: Documents Are Unsearchable

Organizations accumulate vast PDF libraries — research papers, manuals, contracts — that are nearly impossible to query manually. Keyword search misses context; general chatbots lack access to private documents.

Extract

Clean text from diverse PDF structures

Chunk

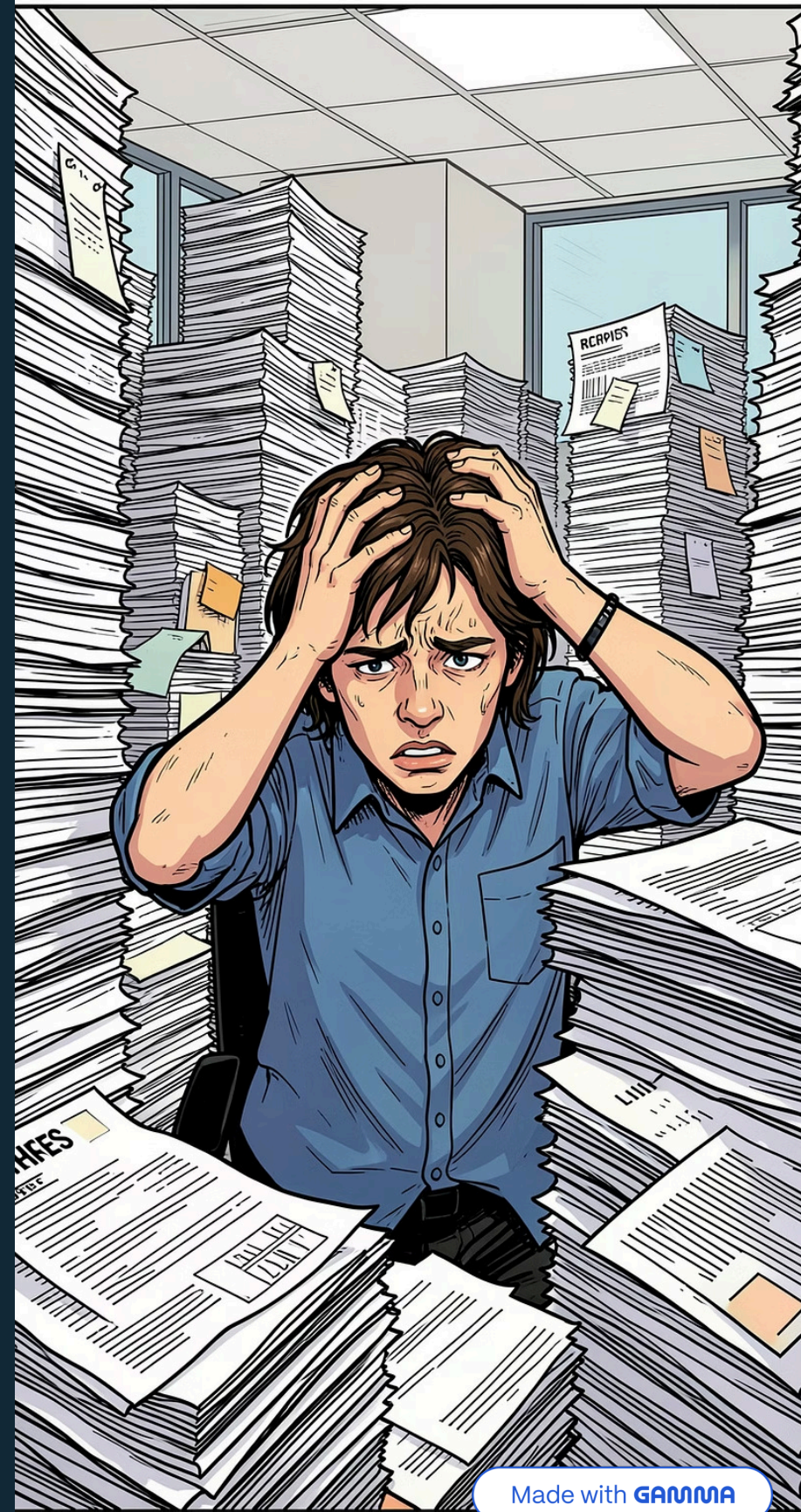
Split into meaningful, searchable segments

Retrieve

Find the most relevant chunks per question

Ground

Generate answers only from retrieved content



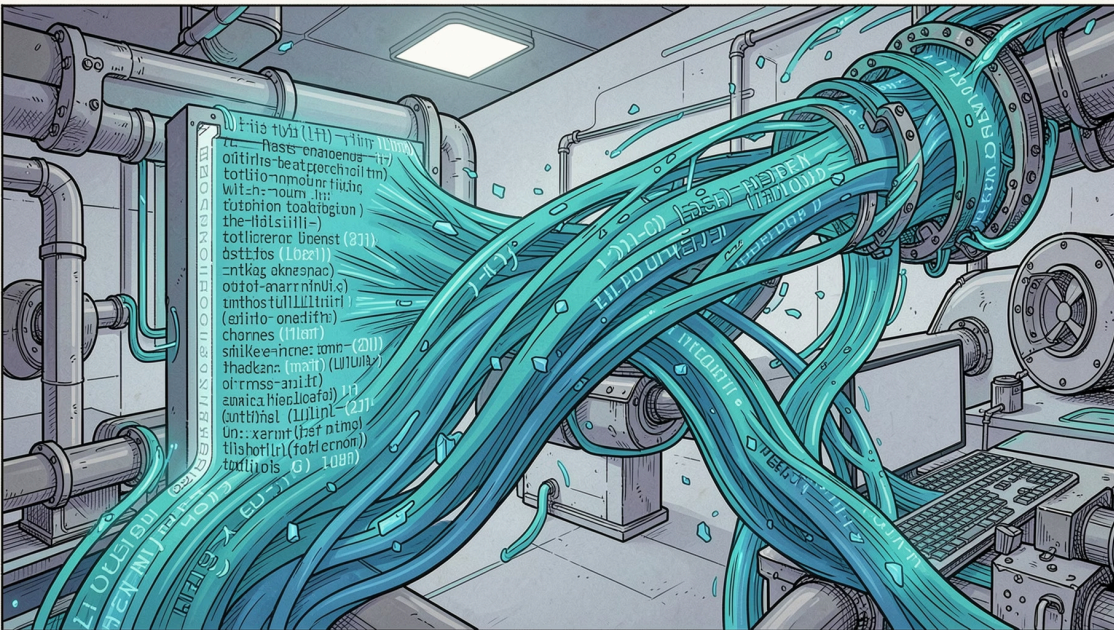
Architecture Overview

The system implements a two-stage RAG pipeline: an **ingestion pipeline** that processes documents once, and a **query pipeline** that runs on every user question.



Every answer is grounded in retrieved document chunks and displayed alongside expandable source citations for full transparency.

Ingestion Pipeline



01

PDF Upload

Users upload one or more PDFs via Streamlit's file uploader

02

Text Extraction

PyPDF2 reads each page and extracts raw text

03

Chunking

Text split into 300-word chunks with 30-word overlap to preserve context across boundaries

04

Embedding & Indexing

TF-IDF vectors built via numpy; stored in FAISS IndexFlatL2 for fast similarity search

Query Pipeline

When a user submits a question, the system retrieves relevant context and generates a grounded answer in under 3 seconds.



Question

User submits via Streamlit chat input



Retrieve

FAISS returns top-4 similar chunks using the shared TF-IDF vocabulary



Generate

Chunks + last 3 turns of history sent to Llama 3.3 70B via Groq API



Answer

Grounded response shown alongside expandable source chunks

Technology Stack

Component	Tool	Purpose
UI	Streamlit	Web interface for upload and chat
PDF Parsing	PyPDF2	Extract text from PDF pages
Chunking	Custom (numpy)	Split text into 300-word chunks
Embedding	TF-IDF (numpy)	Convert text to numerical vectors
Vector DB	FAISS	Store and search embeddings
LLM	Llama 3.3 70B	Generate grounded answers
API	Groq (free tier)	Fast LLM inference

Results

~3s

Inference Time

Typical response time from Groq-hosted Llama 3.3 70B

4

Chunks Retrieved

Top-k nearest neighbors returned per query via FAISS

3

Turns of Memory

Conversation history maintained across chat session

~130

Lines of Python

Clean, single-file implementation covering the full pipeline

- ✓ The system correctly refuses to answer questions outside the document scope — no hallucination, with a clear disclaimer instead.

Concepts Demonstrated



Transformers

Llama 3.3 70B is a transformer-based LLM used for answer generation from retrieved context



Embeddings

Document chunks converted to TF-IDF numerical vectors enabling similarity-based retrieval



Retrieval

FAISS performs efficient nearest-neighbor search to surface the most relevant document chunks



Deployment

Streamlit provides a browser-accessible web interface for upload, chat, and source inspection

Limitations

Technical

Scanned PDFs

PyPDF2 cannot extract text from image-based PDFs — OCR is required

TF-IDF Embeddings

No semantic understanding — "car" and "automobile" are treated as unrelated

No Persistence

FAISS index is lost on page refresh; no session continuity

Scalability

Linear Scan

IndexFlatL2 performs brute-force search — slow at millions of chunks

Rate Limits

Groq free tier caps at ~30 requests/minute

Synchronous UI

Large PDFs block the interface during indexing; no async processing



Roadmap: Path to Production

1

Short-Term

Replace TF-IDF with **Sentence Transformers**; add OCR via Tesseract; persist FAISS index to disk; stream LLM responses token-by-token

2

Medium-Term

Migrate to **Milvus** or **ChromaDB**; add FastAPI backend; implement hybrid BM25 + vector search; support DOCX, TXT, CSV

3

Long-Term

Cloud deploy via **Docker** on **AWS/GCP**; user authentication; cross-encoder re-ranking; RAGAS evaluation pipeline; voice I/O